UNIVERSITAS ISLAM INDONESIA

International Program in Informatics

Midterm Examination (Take Home), Odd Semester AY 2026/2027

Deep Learning

Detecting Players, the Ball, and Referees in Football Broadcast Frames with YOLOv8

A technical feasibility study with transfer learning, source-level data splitting, and a pre-registered evaluation

Student	[Student name]
Student ID (NIM)	[NIM]
Course	Deep Learning
Examiner	Dr. Syarif Hidayat, S.Kom., M.I.T.
Exam date	Wednesday, November 11, 2026
Algorithm chosen	YOLO (YOLOv8n, Ultralytics 8.4.202)

Contents

Executive summary 3
Section 1. Project Description and Case Study Selection 3
1.1 Project objective 3
1.2 Case study 3
1.3 Why YOLO suits this case 3
Section 2. Data Collection and Preprocessing 4
2.1 Candidate datasets and audit 4
2.2 Splitting by source and removing duplicates 4
2.3 Preprocessing methods and rationale 5
2.4 Data risks 7
Section 3. Model Implementation 7
3.1 Model and transfer learning 7
3.2 Architecture 8
3.3 Implementation code 9
3.4 Training configuration 9
Section 4. Model Training and Evaluation 10
4.1 Training 10
4.2 Evaluation protocol 10
4.3 Results 11
Section 5. Results Analysis and Conclusion 15
5.1 In-depth analysis 15
5.2 Were the project objectives achieved? 16
5.3 Recommendations 17
Appendix A. Reproducibility 17

Executive summary

This project trains a YOLOv8n object detector to find **players, the ball, and referees** in football broadcast frames, and measures honestly how well it works on data it has not seen. The model was fine-tuned from COCO weights on 8,451 training images and compared with the same architecture without fine-tuning.

Main results (internal test set of 2,288 images from sources never used for training; thresholds frozen on validation):

- Fine-tuning is clearly better than the pretrained baseline: mean AP@0.5 over the classes the baseline can detect rises from 0.236 to 0.518 (player and ball), and the referee class (absent from COCO) reaches AP@0.5 0.543.
- **Player** (AP@0.5 0.770) and **referee** (0.543) passed the pre-defined success criteria; the model also reaches mAP@0.5 0.595 on an external dataset (RF100).
- The **ball** did not: recall 0.288 and precision 0.521 against targets of 0.50 and 0.70. The weakness is partly underfitting (ball recall is only 0.525 even on training images) and partly a domain gap.
- Speed on a laptop GPU is 22.3 ms/frame and 59.3 ms/frame on CPU (median, batch 1).

Conclusion: detecting players and referees is technically feasible with this setup; **detecting the ball is not yet**, and the report states what evidence supports that and what to do next.

Section 1. Project Description and Case Study Selection

1.1 Project objective

The objective is to build and evaluate a deep-learning detector that localizes three object classes in single frames of football broadcast video: **player**, **ball**, and **referee**. The project is framed as a **technical feasibility study**: the question is whether a small, publicly trainable YOLO model, trained on public data on one laptop, reaches useful accuracy, rather than whether a finished product exists.

To avoid judging the results after seeing them, measurable success criteria were fixed before the test set was evaluated (Section 5.2). In scope: detection of the three classes, evaluation, and robustness tests. Out of scope: object tracking, real-time video processing, and a user interface.

1.2 Case study

The case study is **object detection in football broadcast frames**. A detector that finds players, referees, and the ball is the first step of most sports-analytics pipelines (positions, distances, tactical statistics). The case is challenging in a way that is useful for studying deep learning: players are medium-sized, referees look like players with a different shirt colour, and the ball is a very small object that is often blurred or occluded.

1.3 Why YOLO suits this case

Table 1. Algorithm options considered for the case study.

Option	Why it is (not) suitable here
CNN image classifier	Predicts one label per image; cannot say where objects are or count them. Not suitable for localization.
RNN / LSTM / GRU	Designed for sequences (text, time series). Single frames carry no sequence, so these are not the natural fit.
Two-stage detectors (e.g., Faster R-CNN)	Often accurate but slower and heavier to train; unattractive on a laptop with a 6 GB GPU.
YOLO (single-stage)	One forward pass predicts boxes and classes at three scales; anchor-free decoupled head; pretrained COCO weights already know *person* and *sports ball*, so transfer learning needs little data; fast enough for a laptop.

The smallest variant (YOLOv8n) was chosen because the hardware limit is 6 GB of GPU memory and because the study needs several comparable runs within three days. The known trade-off is that small models have limited capacity for tiny objects such as the ball; Section 5 shows that this trade-off matters.

Section 2. Data Collection and Preprocessing

2.1 Candidate datasets and audit

Table 2. Candidate datasets (audit performed 2026-11-05).

Dataset	Source	Images	Classes	License	Audit status
Soccana player/ball/referee v1	Hugging Face (Adit-jain)	11,673 (README claims 25,000)	Player, Ball, Referee	Not stated; derived from 8 Roboflow datasets	Fully audited
soccer-players (RF100)	Hugging Face (Francesco)	163	football, player, referee (+ parent class)	tag "cc" (unspecific)	Fully audited
Roboflow Universe projects	universe.roboflow.com	unknown	ball / player / referee	mostly unverified	Not audited: pages returned HTTP 403 to automated access; download needs an account
SoccerNet-Tracking / GSR	soccer-net.org	200 sequences (paper)	player, goalkeeper, referee, ball	paper lists CC BY 4.0, data terms unverified	Not used (access not checked)

Selected dataset: Soccana v1 as the main data pool, because it is the only candidate large enough (it contains 3,500+ ball boxes) and spans several sources. RF100 is too small to train on (141 balls) and is kept as an **external test set**.

Audit findings (checked on the data, not taken from the description):

- The dataset contains 11,673 images, not the 25,000 stated in its description.
- The class table in the README is in the wrong order (it lists 1 = Referee, 2 = Ball); the `data.yaml` is correct (0 Player, 1 Ball, 2 Referee). This was confirmed by looking at labelled images and by box size (class 1 boxes are tiny).
- Many images are **SAHI slices**: crops of larger frames at sizes 160, 320, 640, 1080, 1280 and 1920x1080 pixels. About 28% of training images contain no objects.
- Images come from 7 source groups (prefixes in the file names), each originally a separate Roboflow dataset. Licenses were not verified.

2.2 Splitting by source and removing duplicates

The dataset ships with a random train/test split. This is unsafe for video frames, because consecutive frames are almost identical. A perceptual-hash (pHash) check showed that **119 of 2,996 images in the original test split (4.0%) had a near-duplicate (Hamming distance <= 4) in the training split**, and every one of the seven sources appeared in both splits. The data was therefore re-split **by source group**, so that whole sources are held out:

Table 3. Source groups and their assignment (counts before duplicate removal).

Source group	Images	Players	Balls	Referees	Assigned to
v3	3,475	14,948	945	1,396	train
v7	3,546	12,377	635	1,084	train
v2_temp	1,290	7,308	503	679	train
spt_v2	140	1,040	50	85	train
tbd_v2	797	3,133	653	351	validation
v12	1,725	8,701	323	939	test
v5_temp	700	8,265	491	823	test

After assigning sources, the test images were hashed again against train and validation. **137 test images (5.6%) were near-duplicates of images from other sources and were removed** before any evaluation (100 had an identical hash), leaving 2,288 test images. This shows that different Roboflow sources share images. Between

train and validation no near-duplicate (distance <= 4) was found. Residual leakage that pHash cannot detect (different crops of one frame) may remain.

Table 4. Final dataset statistics per split.

Split	Images	Empty	Players	Balls	Referees
Train (4 sources)	8,451	2,862	35,673	2,133	3,244
Validation (tbd_v2)	797	0	3,133	653	351
Internal test (v12, v5_temp, de-duplicated)	2,288	638	14,666	706	1,527
External test (RF100)	163	n/a	1,766	141	96

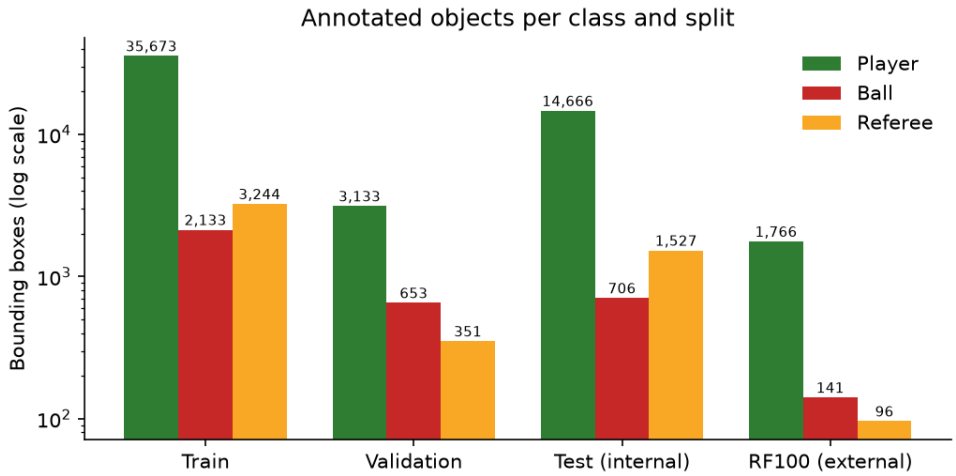


Figure 1. Number of boxes per class and split (log scale). The ball is the rarest class in every split.

The ball is the rarest class: 5.2% of training boxes. All validation images contain a ball, and validation comes from a single source, which turned out to make it easier than the test set (Section 5).

2.3 Preprocessing methods and rationale

Table 5. Preprocessing pipeline.

Step	What was done	Why
Class mapping	Player(0), Ball(1), Referee(2). Goalkeepers are merged into Player by the dataset. RF100: player→Player, football→Ball, referee→Referee, parent class dropped.	Matches the three target classes; verified visually.
Source-level split + de-duplication	Whole sources held out; pHash filter (distance <= 4) on test.	Prevents near-identical frames in train and test, which would inflate scores.
Resize (letterbox)	Images scaled to 640 px on the long side, padded to 640x640 with grey.	Fixed input size; keeps aspect ratio so boxes are not distorted.
Normalization	BGR→RGB, pixel values divided by 255 (range 0–1).	Standard input scale for a network with pretrained weights.
Augmentation (train only)	Mosaic (4 images in one), HSV jitter, random scale ±50%, translation ±10%, horizontal flip (p = 0.5). Mosaic is switched off for the last 10 epochs.	More variety from limited data; mosaic puts small objects at new positions and scales; turning it off at the end lets the model settle on realistic images.
No augmentation on val/test	Evaluation uses plain letterboxed images.	Evaluation must reflect real inputs; augmenting evaluation data would change the question.

No vertical flip and no rotation were used, because broadcast cameras keep the pitch upright. The configuration also contains `auto_augment` and `erasing` entries; in Ultralytics these apply to classification, so they are not counted as detection augmentations here. Augmentation settings are the Ultralytics defaults; they were not tuned, which is a limitation.



Figure 2. A training batch after augmentation (mosaic, scaling, colour jitter), as produced by the training pipeline.

Code 1. Source-level split manifest (scripts/make_split.py)

```
"""Build a split manifest by SOURCE group (not by random frame). Does not modify or copy the original data."""
import pathlib, re, csv, collections
root=pathlib.Path("football_detection/data/work/soccana/V1")
base=lambda s: re.sub(r"_sahi_\d+","",re.sub(r"_\d+$","",s))
SPLIT={"v3":"train","v7":"train","v2_temp":"train","spt_v2":"train","tbd_v2":"val","v12":"test","v5_temp":"test"
    }
rows=[]
for sp in ["train","test"]:
    for p in sorted((root/"images"/sp).glob("*.jpg")):
        b=base(p.stem); rows.append((str(p.relative_to(root)),b,sp,SPLIT[b]))
out=pathlib.Path("football_detection/data/work/splits"); out.mkdir(parents=True,exist_ok=True)
with open(out/"split_by_source_v1.csv","w",newline="") as f:
    w=csv.writer(f); w.writerow(["image","source_group","original_hf_split","new_split"]); w.writerows(rows)
c=collections.Counter(r[3] for r in rows); print(c)
# sanity check: no source group appears in more than one split
g=collections.defaultdict(set)
for r in rows: g[r[1]].add(r[3])
assert all(len(v)==1 for v in g.values()); print("OK: each source group is in exactly one split")
```

Code 2. Duplicate check of the test set against train/validation, run before any test evaluation (scripts/day3_test_prep.py)

```
"""T2: build the internal test list and check pHash leakage of test vs train+val BEFORE any inference. Test
    images with distance <= 4 are excluded."""
import csv, pathlib, collections, json, numpy as np, imagehash
from PIL import Image

W = pathlib.Path("football_detection/data/work"); V1 = (W / "soccana/V1").resolve(); S = W / "splits"


def main():
    rows = list(csv.DictReader(open(S / "split_by_source_v1.csv")))
    sp = collections.defaultdict(list)
    for r in rows: sp[r["new_split"]].append(str(V1 / r["image"]))
    H = lambda ps: np.array([imagehash.phash(Image.open(p).convert("L")).hash.flatten() for p in
        ps]).astype(np.uint8)
    href = H(sp["train"] + sp["val"]); ref = sp["train"] + sp["val"]
    ht = H(sp["test"])
    d = np.array([(href != h).sum(1).min() for h in ht])
    arg = [int((href != h).sum(1).argmin()) for h in ht]
    excl = [i for i in np.where(d <= 4)[0]]
    keep = [p for i, p in enumerate(sp["test"]) if i not in set(excl)]
    (S / "test.txt").write_text("\n".join(keep) + "\n")
```

```
    (S / "test_excluded_near_dup.txt").write_text("\n".join(f"{sp['test'][i]}\t{ref[arg[i]]}\tdist={int(d[i])}"
        for i in excl))
    (S / "data_test.yaml").write_text(f"train: {(S/'train.txt').resolve().as_posix()}\nval:
        {(S/'test.txt').resolve().as_posix()}\n"
                                      "names:\n  0: pemain\n  1: bola\n  2: wasit\n")
    cnt = collections.Counter(); empty = 0; big = 0
    for p in keep:
        lp = pathlib.Path(p.replace("\\", "/").replace("/images/", "/labels/")).with_suffix(".txt")
        rr = [l.split()[0] for l in lp.read_text().splitlines() if l.strip()]
        if not rr: empty += 1
        cnt.update(rr)
        with Image.open(p) as im: big += min(im.size) >= 640
    tr_names = {pathlib.Path(p).name for p in sp["train"] + sp["val"]}
    out = dict(n_test_before=len(sp["test"]), excluded_pHash_le4=len(excl), le8=int((d <= 8).sum()),
        exact=int((d == 0).sum()),
               same_filename_as_trainval=sum(pathlib.Path(p).name in tr_names for p in keep),
               n_test_after=len(keep), empty=empty, min_side_ge640=big, player=cnt["0"], ball=cnt["1"],
                   referee=cnt["2"],
               excluded_by_source=dict(collections.Counter(pathlib.Path(sp["test"][i]).stem.rsplit("_", 1)[0]
                   for i in excl)))
    json.dump(out, open("football_detection/results/day3_test_check.json", "w"), indent=1)
    print(json.dumps(out, indent=1))


if __name__ == "__main__":
    main()
```

Code 3. Augmentation configuration

```
# Ultralytics augmentation / training configuration actually used (args.yaml)
hsv_h: 0.015  hsv_s: 0.7  hsv_v: 0.4
translate: 0.1  scale: 0.5  fliplr: 0.5  flipud: 0.0  degrees: 0.0  shear: 0.0
mosaic: 1.0  close_mosaic: 10  mixup: 0.0
```

2.4 Data risks

- **License unclear** for Soccana (derived from several Roboflow datasets): fine for a course project, not verified for publication.
- **Missing labels:** in a manual check of 20 random validation "false positives", about 19 were real people who had no label (Section 5.1). Precision is therefore probably underestimated.
- **Sources may contain augmented images:** the source v5_temp has 34.3% player boxes that are wider than tall (other sources 0.6–3.3%), and example images are rotated by 90°, which indicates augmentation applied by the original dataset exporter.

Section 3. Model Implementation

3.1 Model and transfer learning

The model is **YOLOv8n** (nano) from Ultralytics. Training starts from weights pretrained on COCO (80 classes) and fine-tunes all layers for the three football classes. The training log reports that 349 of 355 weight tensors were transferred; the remaining six are most likely the final class-prediction layers, which must be re-initialised for 3 classes (inferred, not inspected one by one). The class branch of the head keeps the hidden width (80) of the pretrained model, so the final network has more parameters than a model built from scratch for three classes (3,138,489 versus 3,011,433, both counted from the code).

3.2 Architecture

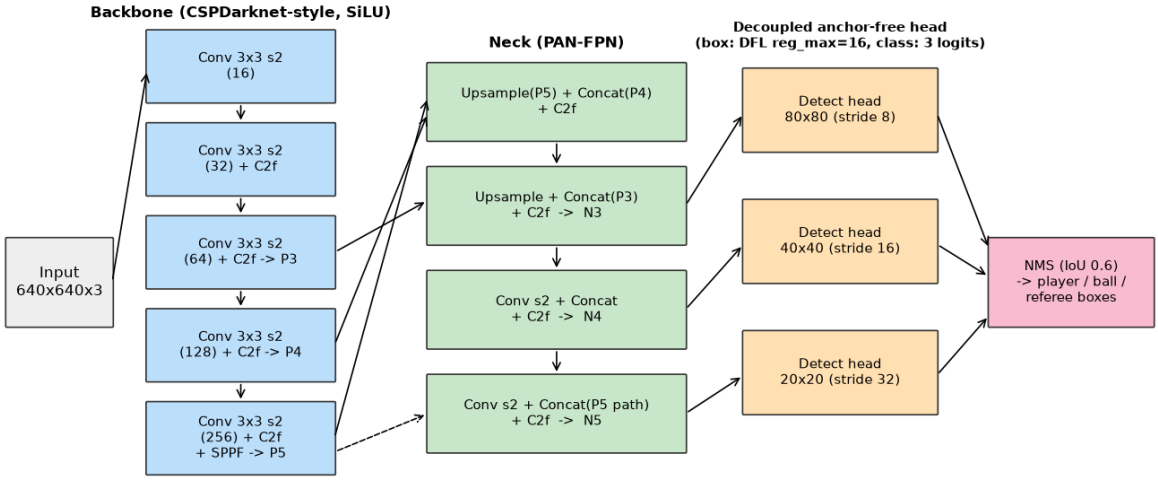


Figure 3. YOLOv8n architecture as used in this project (23 top-level modules; 73 layers and 3,133,193 parameters after Conv–BatchNorm fusion; 8.6 GFLOPs at 640x640).

Table 6. Layer table of the trained model (read from the checkpoint; channels shown after YOLOv8n width scaling).

#	Module	Kernel / stride / out channels	Parameters	Role
0	Conv	3x3 / 2 / 16	464	Stem, downsample x2
1	Conv	3x3 / 2 / 32	4,672	Downsample x4
2	C2f (1 bottleneck)	-- / -- / 32	7,360	Feature extraction
3	Conv	3x3 / 2 / 64	18,560	Downsample x8
4	C2f (2 bottlenecks)	-- / -- / 64	49,664	Output P3 (stride 8)
5	Conv	3x3 / 2 / 128	73,984	Downsample x16
6	C2f (2 bottlenecks)	-- / -- / 128	197,632	Output P4 (stride 16)
7	Conv	3x3 / 2 / 256	295,424	Downsample x32
8	C2f (1 bottleneck)	-- / -- / 256	460,288	Feature extraction
9	SPPF	pool k = 5	164,608	Output P5 (stride 32); multi-scale context
10–12	Upsample, Concat(6), C2f	x2 nearest	148,224	Top-down fusion P5→P4
13–15	Upsample, Concat(4), C2f	x2 nearest	37,248	Top-down fusion → N3 (small objects)
16–18	Conv s2, Concat(12), C2f	3x3 / 2	160,640	Bottom-up fusion → N4
19–21	Conv s2, Concat(9), C2f	3x3 / 2	640,768	Bottom-up fusion → N5
22	Detect	strides 8/16/32, DFL 16 bins, 3 classes	878,953	Decoupled box + class heads
	Total (unfused)		**3,138,489**	

Choices and rationale

- **Convolutions:** 3x3 kernels with stride 2 halve the resolution at each stage (640→320→160→80→40→20), building features from fine to coarse with few parameters.
- **Activation: SiLU** (x·sigmoid(x)) after every convolution with batch normalization. It is smooth and non-saturating for positive values, and it is the activation the pretrained weights were trained with, so transfer learning stays consistent.
- **C2f blocks** (cross-stage partial with extra skip connections): residual-style bottlenecks improve gradient flow while keeping the network small. Repeats are 1, 2, 2, 1 in the backbone (YOLOv8n depth scaling of 0.33).
- **SPPF** (three chained 5x5 max-pools): enlarges the receptive field cheaply so that the deepest features see wide context.
- **PAN-FPN neck:** combines deep (semantic) and shallow (detailed) features top-down and bottom-up. Three output scales (strides 8, 16, 32) let one network handle large players and small balls; the stride-8 map at 80x80 is the main chance for the ball.
- **Decoupled anchor-free head:** separate branches predict box and class. The box uses a distribution over 16 bins per side (Distribution Focal Loss); the class branch has 3 outputs trained with binary cross-entropy.

The box loss is CIoU + DFL, and the label assigner is task-aligned. No anchor sizes need to be tuned for the tiny ball.

- **Loss weights** (box 7.5, class 0.5, DFL 1.5) are the Ultralytics defaults and were not changed.

3.3 Implementation code

The network is defined by the Ultralytics model configuration (`yolov8n.yaml`); training and evaluation code was written for this project. The backbone/head definition below was read from the loaded model; numbers in brackets are the *base* widths that the n-scale (width 0.25) reduces to 16–256 channels.

Code 4. Model definition (yolov8n.yaml, as loaded)

```
backbone:                                    head:
  - [-1, 1, Conv, [64, 3, 2]]    # 0-P1/2      - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [-1, 1, Conv, [128, 3, 2]]   # 1-P2/4      - [[-1, 6], 1, Concat, [1]]
  - [-1, 3, C2f, [128, True]]                  - [-1, 3, C2f, [512]]
  - [-1, 1, Conv, [256, 3, 2]]   # 3-P3/8      - [-1, 1, nn.Upsample, [None, 2, 'nearest']]
  - [-1, 6, C2f, [256, True]]                  - [[-1, 4], 1, Concat, [1]]
  - [-1, 1, Conv, [512, 3, 2]]   # 5-P4/16     - [-1, 3, C2f, [256]]                # N3
  - [-1, 6, C2f, [512, True]]                  - [-1, 1, Conv, [256, 3, 2]]
  - [-1, 1, Conv, [1024, 3, 2]]  # 7-P5/32     - [[-1, 12], 1, Concat, [1]]
  - [-1, 3, C2f, [1024, True]]                 - [-1, 3, C2f, [512]]                # N4
  - [-1, 1, SPPF, [1024, 5]]     # 9           - [-1, 1, Conv, [512, 3, 2]]
                                               - [[-1, 9], 1, Concat, [1]]
                                               - [-1, 3, C2f, [1024]]               # N5
                                               - [[15, 18, 21], 1, Detect, [nc]]    # nc = 3
```

Code 5. Training script (scripts/train.py)

```
"""Train YOLO on the train split, validate ONLY on val. Usage: train.py RUN_NAME INIT_WEIGHTS EPOCHS IMGSZ
    BATCH"""
import sys, time, json, pathlib
from ultralytics import YOLO


def main():
    name, w0, epochs, imgsz, batch = sys.argv[1], sys.argv[2], int(sys.argv[3]), int(sys.argv[4]),
        int(sys.argv[5])
    project = pathlib.Path("football_detection/results/train").resolve()
    assert not (project / name).exists(), "run name already exists; choose a unique name"
    cfg = dict(data=str(pathlib.Path("football_detection/data/work/splits/data_day2.yaml").resolve()),
        epochs=epochs, imgsz=imgsz,
               batch=batch, seed=42, deterministic=True, workers=4, device=0, project=str(project), name=name,
                   exist_ok=False,
               patience=100, plots=True, cache=False)
    t = time.time()
    m = YOLO(w0)
    m.train(**cfg)
    cfg.update(weights_init=w0, train_seconds=round(time.time() - t, 1))
    json.dump(cfg, open(project / name / "run_config.json", "w"), indent=1)
    print("TRAIN_DONE", cfg["train_seconds"])


if __name__ == "__main__":
    main()
```

Code 6. Inference with per-class confidence thresholds

```
# Inference with the frozen per-class thresholds (used for all P/R numbers and figures)
from ultralytics import YOLO
model = YOLO("best.pt")
thr = {0: 0.472, 1: 0.234, 2: 0.280}                # player, ball, referee: from validation
r = model.predict("frame.jpg", imgsz=640, conf=0.001, iou=0.6)[0]
keep = [i for i, (c, s) in enumerate(zip(r.boxes.cls.int().tolist(), r.boxes.conf.tolist())) if s >= thr[c]]
boxes = r.boxes[keep]
```

3.4 Training configuration

Table 7. Training configuration and deviations from the plan.

Setting	Value	Reason / note
Initial weights	yolov8n.pt (COCO)	Transfer learning from a model that already detects people and sports balls
Input size / batch	640 / 8	Batch 16 caused a GPU out-of-memory error on the 6 GB GPU, so it was halved
Epochs	25	Planned 30; 30 epochs would take about 63 minutes (measured 125 s/epoch), above the 1-hour limit per experiment

Optimizer	AdamW, lr0 0.001429, lrf 0.01, momentum 0.937, weight decay 0.0005	Chosen by Ultralytics' automatic rule (`optimizer=auto`) for a short schedule; not tuned
Warm-up	3 epochs	Default; stabilises the first updates
Mixed precision / seed	AMP on / 42, deterministic	Speed and memory; repeatability
Hardware	RTX 3050 Laptop 6 GB, Ryzen 7 8845HS, 13.8 GB RAM	Python 3.12.6, PyTorch 2.5.1+cu121, Ultralytics 8.4.202
Interruption	System RAM ran out at epoch 12	Training was resumed from the saved checkpoint with 2 data-loader workers (resume keeps the schedule but not bit-identical data order)

Section 4. Model Training and Evaluation

4.1 Training

Training used only the training split; validation images were used after every epoch to monitor progress and to choose the checkpoint (`best.pt`). Total training time was about **54 minutes** for 25 epochs (excluding start-up and the crash). The final-epoch validation mAP@0.5 reported by the trainer is 0.837.

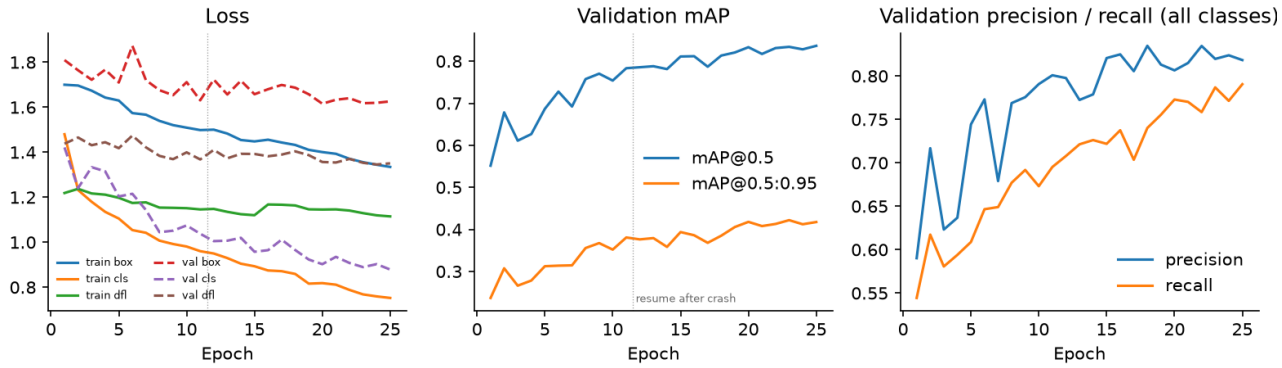


Figure 4. Training and validation losses, validation mAP, and precision/recall per epoch. The dotted line marks the resume after the crash.

Reading the curves. All training losses decrease steadily, validation losses decrease more slowly, and validation mAP is still creeping upward at epoch 25. The model has therefore not clearly converged: more training would probably help a little. The gap between the training class loss (falling fastest) and the validation class loss is the first sign of a train/validation difference.

4.2 Evaluation protocol

- **Matching:** a prediction is correct if its class matches and IoU with a ground-truth box is at least 0.5 (AP@0.5); mAP@0.5:0.95 averages IoU thresholds 0.50–0.95.
- **AP** is computed over all predictions with confidence >= 0.001. **Precision and recall** are reported at one **frozen confidence threshold per class** (maximum-F1 point on validation: player 0.472, ball 0.234, referee 0.280), then applied unchanged to the test sets.
- **Baseline A** is the same YOLOv8n architecture with COCO weights and no fine-tuning: *person* is mapped to Player and *sports ball* to Ball. COCO has no referee class, so the referee is **not available** for the baseline; its mAP is averaged over two classes and is not directly comparable with the three-class mAP of Model B.
- **Speed:** milliseconds per frame (pre-processing + inference + NMS) at batch 1 after warm-up, 100 test images on GPU, 40 on CPU.
- **Order of use:** configuration was frozen on validation, then the test sets were evaluated once. Nothing was changed afterwards.

Code 7. Matching and metric computation (scripts/eval_lib.py, shortened to the core functions)

```
def _pairs(gb, gc, pb, pc, t):
    """Greedy IoU matching (Ultralytics style). Returns an array of [gt_idx, pred_idx]."""
    if len(pb) == 0 or len(gb) == 0: return np.zeros((0, 2), int)
    iou = box_iou(torch.tensor(gb, dtype=torch.float32), torch.tensor(pb, dtype=torch.float32)).numpy()
    iou = iou * (gc[:, None] == pc[None, :])
```

```
    m = np.array(np.nonzero(iou >= t)).T
    if not len(m): return np.zeros((0, 2), int)
    m = m[np.argsort(-iou[m[:, 0], m[:, 1]])]
    m = m[np.unique(m[:, 1], return_index=True)[1]]  # one match per prediction (predictions are sorted by
        confidence)
    m = m[np.unique(m[:, 0], return_index=True)[1]]  # one match per ground truth (same behaviour as
        Ultralytics)
    return m.astype(int)

def match(gb, gc, pb, pc):
    correct = np.zeros((len(pb), len(IOUV)), bool)
    for i, t in enumerate(IOUV):
        m = _pairs(gb, gc, pb, pc, t)
        if len(m): correct[m[:, 1], i] = True
    return correct

def metrics(per, op_conf=None, classes_expected=(0, 1, 2)):
    """Per-class AP (conf 0.001), per-class F1-optimal confidence, and P/R at op_conf {class: threshold} if
        given."""
    tp, conf, pc, tc = _cat(per)
    out = ap_per_class(tp, conf, pc, tc, plot=False)
    ap, uc, f1c, x = out[5], out[6], out[9], out[10]
    res = {}
    for k in classes_expected:
        n_gt = int((tc == k).sum()); n_pred = int((pc == k).sum())
        if k in uc and n_pred > 0:
            j = list(uc).index(k)
            res[NAMES[k]] = dict(ap50=float(ap[j, 0]), ap50_95=float(ap[j].mean()), n_gt=n_gt,
                                 f1_opt_conf=float(x[int(np.argmax(f1c[j]))]), f1_max=float(f1c[j].max()))
        else:
            res[NAMES[k]] = dict(ap50=None, ap50_95=None, n_gt=n_gt, f1_opt_conf=None, f1_max=None)
        if op_conf is not None and op_conf.get(NAMES[k]) is not None:
            t = op_conf[NAMES[k]]
            msk = (pc == k) & (conf >= t)
            TP = int(tp[msk, 0].sum())
            res[NAMES[k]].update(op_conf=t, precision=TP / max(int(msk.sum()), 1) if msk.sum() else None,
                                 recall=TP / n_gt if n_gt else None, tp=TP, n_pred_at_op=int(msk.sum()))
    av = [res[NAMES[k]]["ap50"] for k in classes_expected if res[NAMES[k]]["ap50"] is not None]
    av95 = [res[NAMES[k]]["ap50_95"] for k in classes_expected if res[NAMES[k]]["ap50_95"] is not None]
    res["_mean"] = dict(ap50=float(np.mean(av)) if av else None, ap50_95=float(np.mean(av95)) if av95 else None,
         n_classes=len(av))
    return res
```

4.3 Results

Table 8. Validation set (797 images). Used for model and threshold selection.

Class	A: AP@0.5	A: P / R	B: AP@0.5	B: P / R	B: AP@0.5:0.95
Player	0.638	0.648 / 0.664	0.878	0.828 / 0.879	0.460
Ball	0.381	0.554 / 0.391	0.781	0.827 / 0.734	0.384
Referee	n/a	n/a	0.810	0.792 / 0.801	0.391
mAP@0.5	0.510 (2 classes)		**0.823** (3 classes)		0.412

Table 9. Internal test set (2,288 images, unseen sources). Final result.

Class	A: AP@0.5	A: P / R	B: AP@0.5	B: P / R	B: AP@0.5:0.95
Player	0.394	0.675 / 0.355	0.770	0.912 / 0.680	0.438
Ball	0.078	0.342 / 0.092	0.267	0.521 / 0.288	0.098
Referee	n/a	n/a	0.543	0.776 / 0.531	0.281
mAP@0.5	0.236 (2 classes)		**0.527** (3 classes)		0.272

Table 10. External test set RF100 (163 images; only 141 balls and 96 referees, so these numbers are noisy).

Class	A: AP@0.5	A: P / R	B: AP@0.5	B: P / R	B: AP@0.5:0.95
Player	0.402	0.810 / 0.232	0.954	0.956 / 0.890	0.412
Ball	0.013	0.190 / 0.028	0.222	0.300 / 0.191	0.054
Referee	n/a	n/a	0.608	0.347 / 0.906	0.244
mAP@0.5	0.208 (2 classes)		**0.595** (3 classes)		0.237

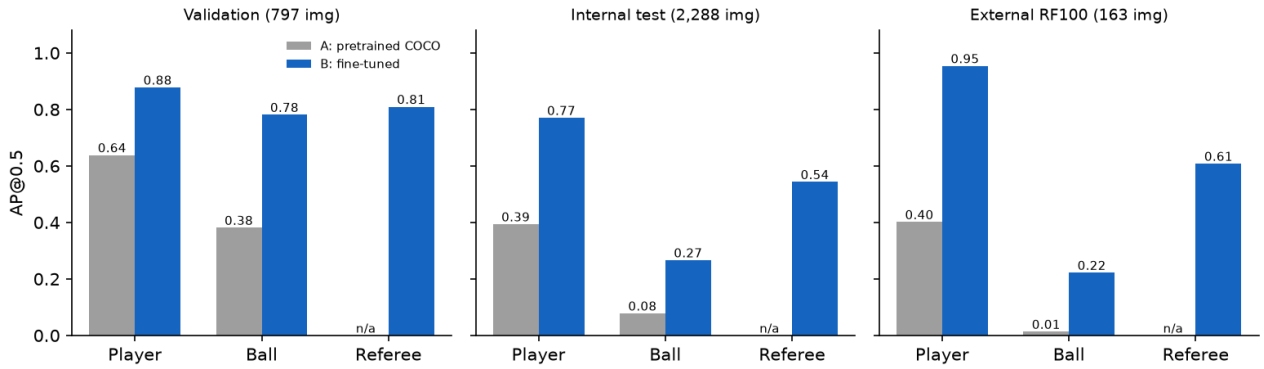


Figure 5. AP@0.5 per class for Baseline A (grey) and Model B (blue) on validation, internal test and external data.

Uncertainty. A bootstrap over test images (1,000 resamples, seed 42) gives 95% intervals for Model B on the internal test set: mAP@0.5 0.507–0.548, ball AP@0.5 0.232–0.304, ball recall 0.258–0.321. Resampling is per image and therefore does not capture dependence between slices of the same frame, so the true uncertainty is probably wider.

Performance on training data versus unseen data

To see how much the model has learned versus how well it generalises, the frozen configuration was also applied to a random subset of 1,000 training images (without augmentation, seed 42).

Table 11. Model B on seen and unseen data.

Data	Player AP@0.5	Ball AP@0.5	Referee AP@0.5	Ball recall	mAP@0.5
Train subset (1,000)	0.947	0.563	0.783	0.525	0.764
Validation (797)	0.878	0.781	0.810	0.734	0.823
Internal test (2,288)	0.770	0.267	0.543	0.288	0.527
RF100 (163)	0.954	0.222	0.608	0.191	0.595

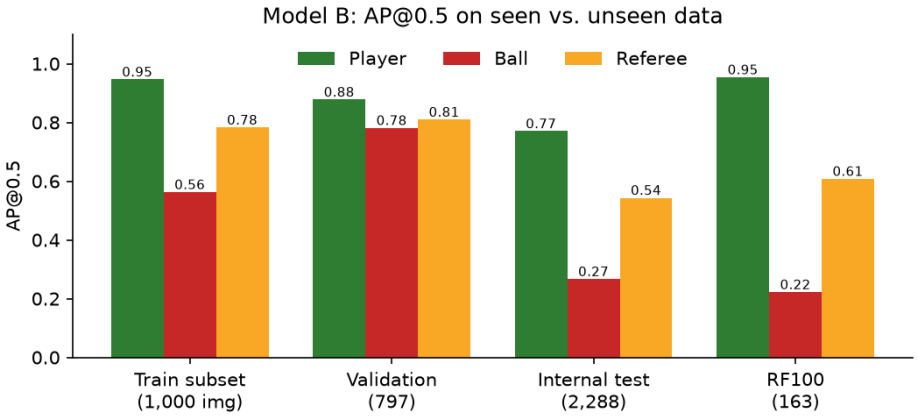


Figure 6. Model B AP@0.5 per class on seen (train subset) and unseen data.

Interpretation. For players, performance on unseen sources is lower than on training images but still good (AP@0.5 0.77 on test vs 0.95 on train), which is an ordinary generalisation gap. For the ball, even the **training** images give only AP@0.5 0.563 and recall 0.525: the model has not learned to find the ball well even where it has seen the answers. This points to **underfitting** (small model, 25 epochs, tiny objects) as the first problem, and domain difference as the second. Validation is higher than training for the ball because the validation source (tbd_v2) is easy: all of its images contain a ball.

Confusion matrices and precision–recall curves

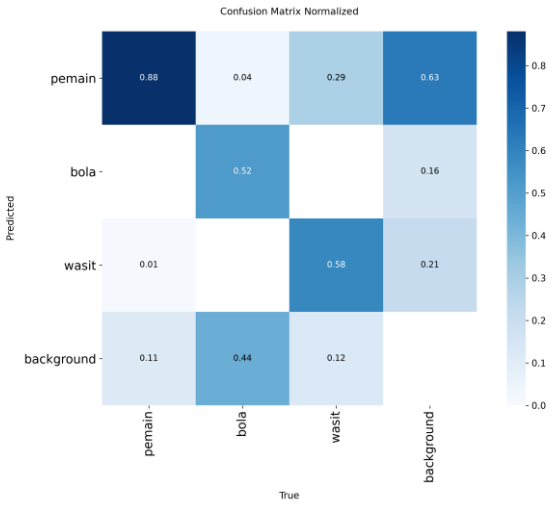


Figure 7. Normalized confusion matrix of Model B on the internal test set (Ultralytics default threshold 0.25; background = missed or false detections).

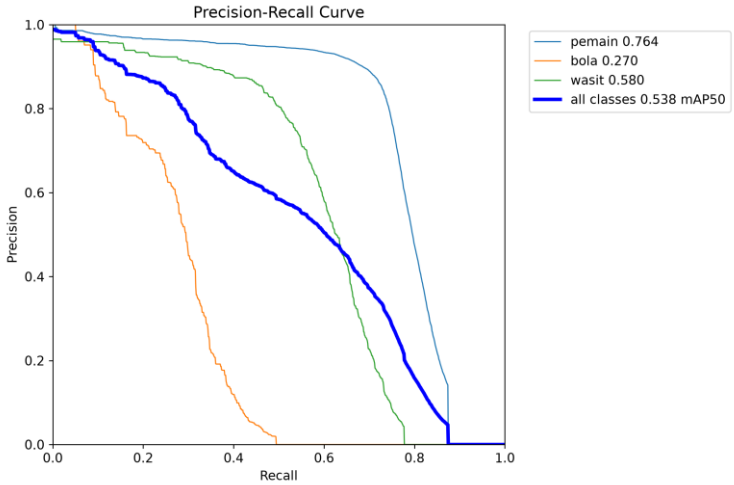


Figure 8. Precision–recall curves of Model B on the internal test set; the ball curve is clearly the lowest.

Speed

Table 12. Inference speed on the laptop (batch 1, 640x640).

Model	GPU median (p95) ms/frame	CPU median (p95) ms/frame
A: pretrained (test images)	16.7 (20.9)	43.5 (47.7)
B: fine-tuned (test images)	22.3 (26.9)	59.3 (65.6)
B: fine-tuned (validation images, earlier run)	14.4 (19.0)	48.6 (57.3)

A and B share the same architecture, so the difference between A and B in the table is measurement variation (the laptop's load changed between runs; the same model measured 14.4 ms earlier), not a real speed difference.

Robustness to image degradations

Each degradation was applied in memory to a copy of the test images (the files were not changed), one at a time, with levels fixed before the run. Model B and its frozen thresholds were used.

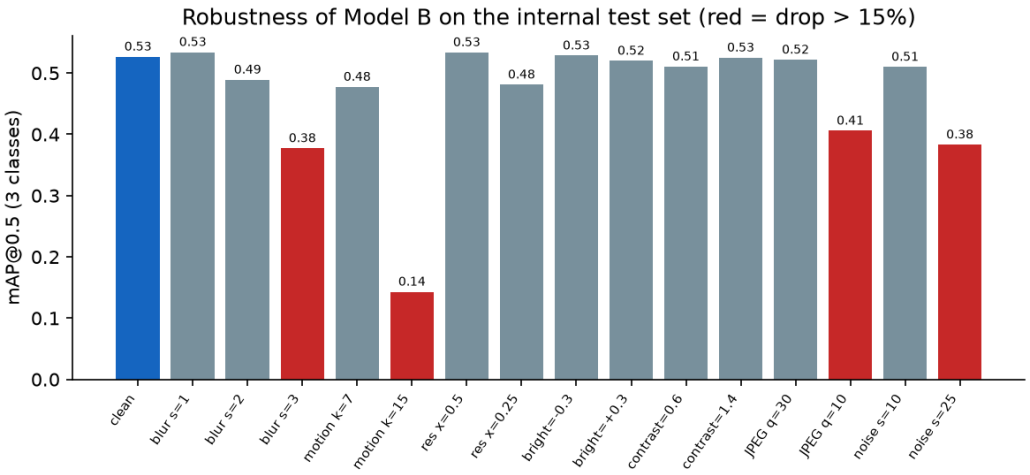


Figure 9. mAP@0.5 of Model B under degradations on the internal test set. Red bars mark a drop of more than 15% relative to clean images.

Table 13. Robustness results (negative drop = slight improvement).

Degradation	mAP@0.5	Relative drop	Ball recall
bersih	0.527	0.0%	0.288
gauss_sigma=1	0.534	-1.4%	0.280
gauss_sigma=2	0.489	7.1%	0.242
gauss_sigma=3	0.377	28.4%	0.133
motion_kernel=7	0.477	9.4%	0.212
motion_kernel=15	0.143	72.8%	0.054
down_up=0.5	0.534	-1.3%	0.285
down_up=0.25	0.481	8.7%	0.228
brightness=-0.3	0.529	-0.3%	0.285
brightness=+0.3	0.520	1.3%	0.290
contrast=0.6	0.511	3.0%	0.279
contrast=1.4	0.525	0.3%	0.293
jpeg_quality=30	0.522	0.9%	0.261
jpeg_quality=10	0.406	22.9%	0.125
gauss_noise_sigma=10	0.510	3.2%	0.261
gauss_noise_sigma=25	0.383	27.2%	0.089

Mild and medium degradations (blur sigma <= 2, half resolution, brightness ±30%, contrast, JPEG quality 30, noise sigma 10) change mAP by less than 10%. Strong ones (motion blur k = 15, blur sigma 3, JPEG quality 10, noise sigma 25) cost 23–73%, and the ball recall collapses to 0.05–0.13. The "half resolution" test hardly matters, probably because many test images are small slices that are enlarged to 640 px anyway, so this test understates the effect of real low-resolution video.

Code 8. Image degradations (scripts/robustness.py, excerpt)

```
def gauss(img, s): return cv2.GaussianBlur(img, (0, 0), s)

def motion(img, k):
    ker = np.zeros((k, k), np.float32); ker[k // 2, :] = 1.0 / k
    return cv2.filter2D(img, -1, ker)

def down_up(img, f):
    h, w = img.shape[:2]
    small = cv2.resize(img, (max(1, int(w * f)), max(1, int(h * f))), interpolation=cv2.INTER_AREA)
    return cv2.resize(small, (w, h), interpolation=cv2.INTER_LINEAR)

def bright(img, b): return np.clip(img.astype(np.float32) * (1 + b), 0, 255).astype(np.uint8)

def jpeg(img, q): return cv2.imdecode(cv2.imencode(".jpg", img, [cv2.IMWRITE_JPEG_QUALITY, q])[1],
    cv2.IMREAD_COLOR)
```

Section 5. Results Analysis and Conclusion

5.1 In-depth analysis

Strengths

- **Transfer learning pays off.** Compared with the same network without fine-tuning, AP@0.5 for players rises by +0.376 and for the ball by +0.188 on the test set, and a new class (referee) becomes available at AP@0.5 0.543.
- **Players are detected reliably:** AP@0.5 0.770 on unseen sources and 0.954 on the external dataset, with precision 0.912.
- **Good tolerance to common degradations** (brightness, contrast, moderate blur, reduced resolution).
- **Fast and small:** 22 ms/frame on the laptop GPU and about 59 ms on its CPU, with a 6.5 MB weight file.

Limitations and their causes

- **The ball is the main failure.** On the test set recall is 0.288 (95% interval 0.26–0.32) and precision 0.521. Both small and large balls are missed (recall 0.259 and 0.331), so size alone does not explain it, and restricting to images of at least 640 px gives about the same result (ball AP@0.5 0.249).
- **Underfitting:** ball recall on training images is only 0.525; training curves are still improving at epoch 25; the model is the smallest variant.
- **Domain gap and data quality:** validation (one easy source) gave ball recall 0.73, the test set 0.29. The test source v5_temp holds 63% of the test balls, shows signs of exporter augmentation, and gives much lower scores than v12:

Table 14. Model B per test source (descriptive; not used for any decision).

Test source	Images	mAP@0.5	Player AP@0.5	Ball AP@0.5	Ball P / R	Referee AP@0.5
v12	1,646	0.712	0.974	0.421	0.694 / 0.414	0.740
v5_temp	642	0.366	0.550	0.198	0.403 / 0.212	0.351

- **Referee vs player confusion:** 153 referees were predicted as players on the test set (59 players as referees). Referees differ mostly by shirt colour, which varies between matches. On RF100, referee precision is low (0.35) while recall is high (0.91): in that domain many non-referees are called referees.
- **Label problems:** in a manual review of 20 random validation "false positives" (player class), about 19 were real people without a label. Reported precision for players is therefore probably lower than the true value; the size of the effect was not measured.

Error analysis (internal test set)

Table 15. Errors of Model B on the internal test set at the frozen thresholds (IoU 0.5; automatic grouping, so categories can overlap).

Error type	Count
Missed player	3,921
Poorly localized player (IoU 0.1–0.5)	1,364
Missed referee	523
Missed ball	423
Poorly localized ball	165
Referee predicted as player	153
Poorly localized referee	136
Background predicted as player	126
Background predicted as ball	102
Background predicted as referee	74
Player predicted as referee	59
Duplicate player	27
Duplicate referee	5
Total	**7,078**

Most errors are **objects the model missed**, not false alarms. Of the 706 test balls, 423 were missed and 165 were found but poorly localized. In sampled images, missed balls were very small, blurred, in the goal net, or against a similar background; missed players were often small or rotated (in the augmented source).

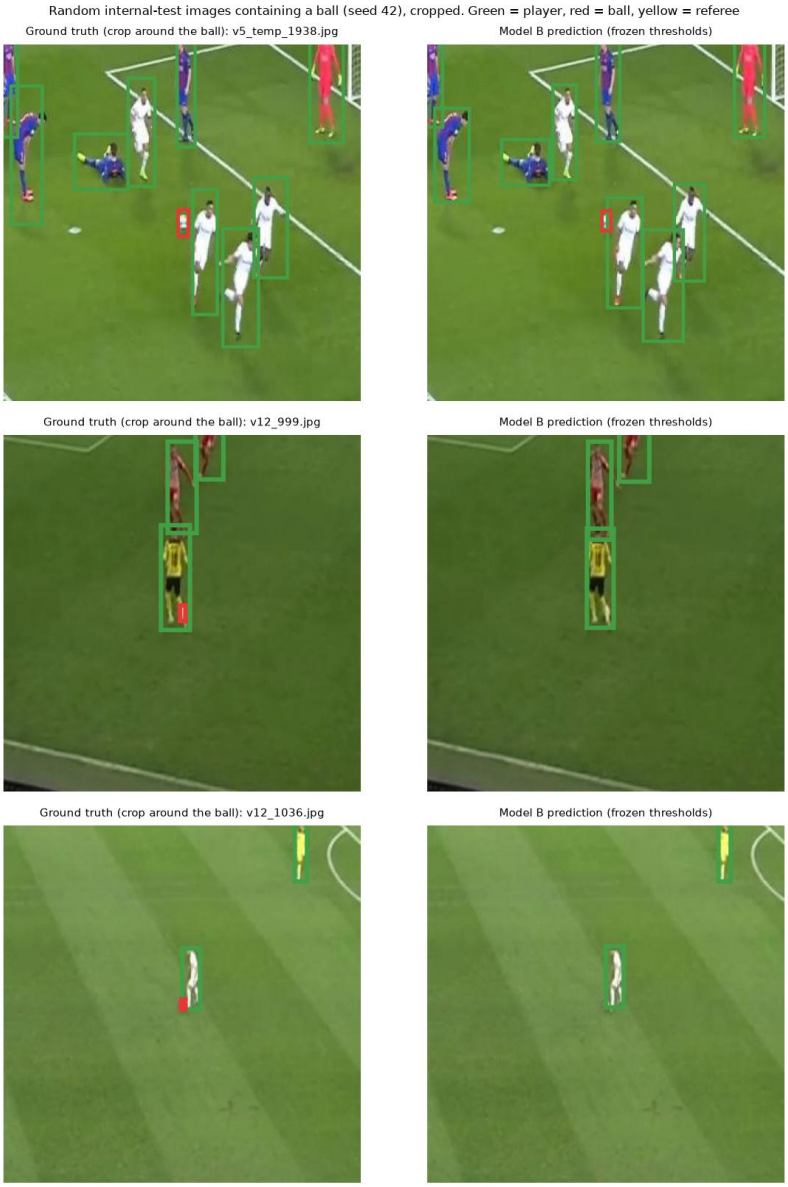


Figure 10. Three random test images that contain a ball, cropped around it: ground truth (left) and Model B prediction (right). The ball is found in the first image and missed in the other two.

5.2 Were the project objectives achieved?

The success criteria were set before the test sets were evaluated and not changed afterwards. They are judgement-based targets for a feasibility study, not an industry standard, and they were chosen after seeing validation results, so they may favour achievable values. Each row is judged exactly as it came out.

Table 16. Pre-defined success criteria and outcomes (criteria 1–5 and 8 are core; 6–7 are reported as risks).

#	Criterion	Result	Target	Outcome
1	Player AP@0.5 (internal test)	0.770	≥ 0.7	**PASS**
2	Referee AP@0.5 (internal test)	0.543	≥ 0.5	**PASS**
3	Ball recall (internal test, frozen threshold)	0.288	≥ 0.5	**FAIL**
4	Ball precision (internal test, frozen threshold)	0.521	≥ 0.7	**FAIL**
5	mAP@0.5, 3 classes (external RF100)	0.595	≥ 0.5	**PASS**
6a	Relative mAP@0.5 drop, Gaussian blur sigma = 2	7.1%	≤ 25%	**PASS**
6b	Relative mAP@0.5 drop, half resolution	-1.4%	≤ 25%	**PASS**

6c	Relative mAP@0.5 drop, brightness ±30% (worse of the two)	1.3%	≤ 15%	**PASS**
7a	Median ms/frame on GPU	22.3 ms	≤ 50 ms	**PASS**
7b	Median ms/frame on CPU	59.3 ms	≤ 200 ms	**PASS**
8a	Player AP@0.5 gain, B minus A (internal test)	0.376	≥ 0.1	**PASS**
8b	Ball AP@0.5 gain, B minus A (internal test)	0.188	≥ 0.1	**PASS**

Verdict. Feasibility is proven for **players and referees** (criteria 1, 2, 5, 8a pass) and for the claim that **fine-tuning clearly beats the pretrained model**. It is **not proven for the ball** (criteria 3 and 4 fail), so the overall objective of detecting all three classes is **only partially achieved**. Speed and robustness targets pass, but the speed target does not claim real-time processing and the robustness test on resolution is weak, as discussed above.

5.3 Recommendations

Ordered by the evidence in this report; each is a hypothesis to test, one change at a time, with a **new test set**, because the current one has been used.

- **Fix the data before the model.** Use a cleaner, multi-source ball dataset (for example SoccerNet with a split by match, if access is allowed), remove or flag sources with exporter augmentation, and review missing labels. Build a validation set from several sources, including frames without a ball.
- **Give the ball more resolution:** train and run at 960–1280 px, or run the detector on overlapping tiles (SAHI-style inference). Ball recall is low even for large balls, so resolution is not the only lever.
- **Give the model more capacity and time:** try YOLOv8s/m and more epochs on a larger GPU (for example Kaggle or Colab); curves were still improving and ball recall on training data is only 0.53.
- **Use time:** a tracker or temporal smoothing across frames could recover balls missed in single frames (outside the scope of this project).
- **Report confidence intervals and per-source results** by default, and check the referee class on more matches, because shirt colours differ.
- **Clarify licenses** before any publication of the model or data.

Overall conclusion: a small YOLO model fine-tuned on public data detects players and referees well enough on unseen sources, but it does not yet detect the ball reliably. The most valuable lesson of the project is methodological: a random split would have hidden both the leakage and the weak ball performance, while a source-level split with a frozen protocol exposed them.

Appendix A. Reproducibility

Item	Value
Random seed	42 (training, sampling, bootstrap)
Model B weights	results/train/B_yolov8n_e25_s42_b8/weights/best.pt, 6,506,915 bytes
SHA-256	a189bcdb3fd729dd2885b1f7e494bd084c3013b304eb4b221a4218f5f4d8a542
Frozen thresholds (Model B)	player 0.472, ball 0.234, referee 0.280
Frozen thresholds (Baseline A)	player 0.338, ball 0.146
Ball size boundary	normalized box area 0.000361 (median of training balls)
Scripts	scripts/: make_split.py, day2_data_check.py, train.py, resume.py, eval_lib.py, day3_freeze.py, day3_test_prep.py, rf100_to_yolo.py, day3_eval.py, robustness.py, error_analysis.py, day3_criteria.py, report_train_eval.py, report_figures.py, build_report.py
Result files	results/summary.csv, results/runs/*.json, results/day3_robustness.csv, results/day3_criteria.json